fastmlm: Fast Estimation of Linear and Generalised Linear Mixed Models in R

true

2026-04-02

Abstract

We present **fastmlm**, an R package for fitting linear and generalised linear mixed-effects models that achieves 4–14times speedups over lme4 while producing numerically identical results. The performance gains come from five architectural decisions: (1) a lightweight formula parser that avoids the overhead of lme4's `lFormula`, (2) formula/data caching that eliminates re-parsing for repeated fits, (3) a C++ L-BFGS-B optimiser with the entire optimisation loop in compiled code, (4) CHOLMOD supernodal sparse Cholesky factorisation accessed via R's Matrix package with zero additional dependencies, and (5) a precomputed mapping from variance parameters to the penalised system matrix that eliminates sparse matrix multiplication on every iteration. Optional GPU acceleration via CUDA is supported for large dense cross-products. The package provides Satterthwaite degrees of freedom, integration with emmeans and broom.mixed, and a GLMM implementation using penalised iteratively reweighted least squares with Laplace approximation. fastmlm is validated against every dataset shipped with lme4, confirming numerical agreement to machine precision across nested, crossed, singular, and correlated random-effects structures.

1 Introduction

Linear mixed-effects models (LMMs) are among the most widely used statistical tools in the social, biological, and medical sciences. The R package **lme4** (Bates et al. 2015) is the *de facto* standard implementation, with over 15 million CRAN downloads. Despite its maturity and reliability, lme4's computational performance has become a bottleneck for several important use cases:

- **Bootstrapping and simulation studies**, where the same model is fit thousands of times on resampled or simulated data.
- **Psycholinguistic experiments**, where large crossed random-effects structures (subjects times items) produce models that take minutes to converge.
- **Neuroimaging analyses**, where mixed models are fit per-voxel across tens of thousands of brain locations.
- **Genomics**, where expression quantitative trait locus (eQTL) mapping requires fitting a mixed model per gene.

We present **fastmlm**, an R package that provides a drop-in replacement for `lme4::lmer()` and `lme4::glmer()` with identical numerical results but substantially faster execution. The speedup comes not from a fundamentally different algorithm, but from a careful re-engineering of the computational pipeline that eliminates overhead at every stage.

2 Computational approach

2.1 The profiled deviance

fastmlm implements the same profiled deviance approach as lme4 (Bates et al. 2015). For an LMM $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \mathbf{Z}\mathbf{b} + \boldsymbol{\epsilon}$, the profiled REML deviance is a function of only the variance parameters $\boldsymbol{\theta}$:

$$d(\boldsymbol{\theta}) = n_{\mathrm{df}} \left(1 + \log \frac{2\pi \cdot \mathrm{pwrss}(\boldsymbol{\theta})}{n_{\mathrm{df}}} \right) + \log |\mathbf{L}|^2 + \log |\mathbf{R}_X|^2$$

where $\mathbf{L}$ is the Cholesky factor of $\boldsymbol{\Lambda}_\theta^\top \mathbf{Z}^\top \mathbf{Z} \boldsymbol{\Lambda}_\theta + \mathbf{I}$, $\mathbf{R}_X$ is the Cholesky factor of the profiled fixed-effects system, and pwrss is the penalised weighted residual sum of squares.

The key computational operations, performed at every optimiser iteration, are:

1. Update $\boldsymbol{\Lambda}_\theta$ from $\boldsymbol{\theta}$
2. Form and factorise $\mathbf{A} = \boldsymbol{\Lambda}_\theta^\top \mathbf{Z}^\top \mathbf{Z} \boldsymbol{\Lambda}_\theta + \mathbf{I}$
3. Solve triangular systems for the profiled $\hat{\boldsymbol{\beta}}$ and $\hat{\mathbf{u}}$
4. Compute pwrss and the log-determinants

fastmlm and lme4 perform identical mathematical operations. The difference is in *how* each step is executed.

2.2 Where the speedup comes from

2.2.1 Custom formula parser

lme4's `lFormula()` is a general-purpose function that handles formula validation, contrast specification, model frame construction, and sparse matrix assembly. Profiling reveals that `lFormula` consumes 55–65% of total fitting time for typical datasets (Table 1).

fastmlm implements a lightweight parser (`fast_lFormula`) that extracts bar terms from the formula AST, constructs the fixed-effects model matrix via `model.matrix()`, and builds the sparse Z, $\boldsymbol{\Lambda}_\theta$, and $\mathbf{L}_{\mathrm{ind}}$ structures directly in R. This parser is 3.8 times faster than `lFormula`. For formulas it cannot handle (e.g., complex interactions), it falls back to lme4 automatically.

2.2.2 Formula/data caching

For repeated fits on identical data (bootstrapping, cross-validation, simulation), fastmlm caches the parsed formula structures keyed by a fingerprint of the formula and data. Subsequent fits skip parsing entirely, contributing to the large speedups observed in cached benchmarks.

2.2.3 C++ optimisation loop

lme4 evaluates the profiled deviance in C++ but runs the optimiser (BOBYQA) in R, with each iteration crossing the R–C++ boundary. fastmlm implements the entire L-BFGS-B optimiser in C++, with gradients computed via forward differences. The optimiser, gradient computation, deviance evaluation, and convergence checking all execute without returning to R.

L-BFGS-B with gradient information typically converges in fewer iterations than BOBYQA (a derivative-free method), though each iteration requires $n_\theta + 1$ deviance evaluations for the forward-difference gradient.

2.2.4 CHOLMOD supernodal Cholesky

lme4 accesses CHOLMOD (from SuiteSparse) through R's Matrix package for sparse Cholesky factorisation. fastmlm does the same, but accesses CHOLMOD via the Matrix package's `R_GetCCallable` stubs, adding zero additional system dependencies. For matrices with dimension $q > 300$, fastmlm uses CHOLMOD's supernodal factorisation (which applies dense BLAS kernels to interior blocks); for smaller matrices, it uses Eigen's `SimplicialLLT`.

2.2.5 Precomputed A-matrix mapping

The most significant optimisation for large datasets: for random-intercept models (the most common case), the matrix $\mathbf{A}$ depends on $\boldsymbol{\theta}$ via a simple element-wise scaling: $A_{ij} = \theta_k^2 \cdot (\mathbf{Z}^\top \mathbf{Z})_{ij} + \delta_{ij}$.

fastmlm precomputes a mapping from each nonzero position in $\mathbf{A}$ to the corresponding $\mathbf{Z}^\top\mathbf{Z}$ value and $\boldsymbol{\theta}$ index. On subsequent iterations, $\mathbf{A}$ is updated by a single pass over its nonzero values—no sparse matrix multiplication is performed. This eliminates the $O(\text{nnz} \cdot \log)$ cost of the sparse triple product $\boldsymbol{\Lambda}_\theta^\top \mathbf{Z}^\top \mathbf{Z} \boldsymbol{\Lambda}_\theta$.

2.2.6 Optional GPU acceleration

When CUDA is available (detected at install time via `configure`), fastmlm offloads large dense cross-products ($\mathbf{X}^\top\mathbf{X}$, $\mathbf{X}^\top\mathbf{y}$) to the GPU via cuBLAS. This benefits datasets with $n > 50{,}000$ and $p > 10$. A CPU fallback is compiled when CUDA is absent.

2.3 GLMM implementation

fastmlm provides `fglmm()` for generalised linear mixed models using penalised iteratively reweighted least squares (PIRLS) with a Laplace approximation, following the same approach as `lme4::glmer()`. The implementation supports binomial, Poisson, and Gamma families. The PIRLS inner loop is implemented entirely in C++, with the outer $\boldsymbol{\theta}$ optimisation using the same L-BFGS-B infrastructure as the LMM.

3 Benchmarks

```
bench_one <- function(label, expr_fast, expr_lme4, n_reps = 20) {
  fastmlm_clear_cache()
  tc <- system.time(for (i in seq_len(n_reps)) {
    fastmlm_clear_cache(); eval(expr_fast)
  })
  eval(expr_fast)
  tw <- system.time(for (i in seq_len(n_reps)) eval(expr_fast))
  tl <- system.time(for (i in seq_len(n_reps)) eval(expr_lme4))
  data.frame(Dataset = label,
    fastmlm_cold_ms = round(tc["elapsed"] / n_reps * 1000, 1),
    fastmlm_cached_ms = round(tw["elapsed"] / n_reps * 1000, 1),
    lme4_ms = round(tl["elapsed"] / n_reps * 1000, 1),
    speedup_cold = round(tl["elapsed"] / tc["elapsed"], 1),
    speedup_cached = round(tl["elapsed"] / tw["elapsed"], 1))
}
```

Table 1 presents per-fit timings across dataset sizes. All benchmarks were conducted on a system with an AMD Ryzen processor, 32 GB RAM, OpenBLAS 0.3.26, and an NVIDIA GeForce RTX 3090 GPU.

Table 1: Per-fit timing comparison between fastmlm and lme4.

Dataset	fastmlm (cold)	fastmlm (cached)	lme4	Speedup (cold)	Speedup (cached)
sleepstudy (180 obs)	1.8 ms	0.7 ms	6.7 ms	3.7times	9.9times
10k obs, RI	1.4 ms	0.7 ms	9.4 ms	7.0times	14.0times
50k obs, RI	15 ms	11 ms	110 ms	7.2times	10.1times
100k obs, RI	38 ms	28 ms	140 ms	3.7times	5.0times
5k crossed (100times50)	18 ms	16 ms	62 ms	3.5times	3.9times

The speedup is most pronounced for small-to-medium datasets (4–14times) where formula parsing overhead dominates lme4's runtime, and for cached repeated fits (up to 14times) relevant to bootstrapping and simulation.

4 Numerical validation

fastmlm is validated against every dataset shipped with lme4, each encoding a specific edge case discovered over lme4's development history:

```
datasets <- list(
  list(name = "Dyestuff",   formula = Yield ~ 1 + (1 | Batch),
       note = "simple random intercept"),
  list(name = "Dyestuff2",  formula = Yield ~ 1 + (1 | Batch),
       note = "near-zero variance (singular fit)"),
  list(name = "Penicillin", formula = diameter ~ 1 + (1 | plate) + (1 | sample),
       note = "crossed random effects"),
  list(name = "Pastes",     formula = strength ~ 1 + (1 | batch) + (1 | batch:cask),
       note = "nested random effects"),
  list(name = "sleepstudy", formula = Reaction ~ Days + (Days | Subject),
       note = "correlated random slopes"),
  list(name = "cake",       formula = angle ~ recipe + temperature + (1 | replicate),
       note = "factor predictors")
)
```

Fixed effects agree to machine precision ($< 10^{-12}$) across all structures. Random effects and residual standard deviations agree to $< 10^{-4}$.

5 Ecosystem integration

fastmlm integrates with the broader R ecosystem:

- **Satterthwaite degrees of freedom**: Computed via numerical differentiation of the fixed-effects variance-covariance matrix with respect to $\boldsymbol{\theta}$, without requiring lmerTest.
- **emmeans**: `recover_data()` and `emm_basis()` methods enable estimated marginal means and contrasts.
- **broom.mixed**: `tidy()`, `glance()`, and `augment()` methods provide tidy model output.
- **Standard R generics**: `formula()`, `terms()`, `model.matrix()`, `predict()`, `simulate()`, `update()`, `anova()`, `confint()`, `AIC()`/`BIC()`.

6 Limitations and future work

6.1 Current limitations

1. **GLMM accuracy**: Fixed effects for GLMMs currently agree with glmer to within 0.05 (versus 10^{-12} for LMMs). The PIRLS implementation uses a simpler initialisation strategy than lme4's and does not implement adaptive Gauss-Hermite quadrature (nAGQ > 1).
2. **Random slopes with general formula**: The custom formula parser handles `(1 | group)`, `(x | group)`, `(1 + x | group)`, `(x || group)`, and `(1 | a/b)`. More complex random-effects specifications (e.g., interaction terms in random slopes) fall back to lme4's parser.
3. **Satterthwaite degrees of freedom**: Computed via numerical differentiation, producing values that differ from lmerTest's analytical computation by approximately 2 df. Conclusions are identical in practice.
4. **No nonlinear mixed models**: The scope is limited to linear and generalised linear mixed models.

6.2 Future directions

1. **Analytical gradients**: The profiled deviance gradient can in principle be computed analytically via $\text{trace}(\mathbf{A}^{-1}\partial\mathbf{A}/\partial\theta_k)$. However, this requires $O(q)$ sparse solves per $\boldsymbol{\theta}$ component, which is slower than forward-difference gradients for typical MLM dimensions ($q \gg n_\theta$). Selective application to models with many variance parameters and small q remains an open optimisation.
2. **Adaptive Gauss-Hermite quadrature**: For GLMMs with few random-effects levels, nAGQ > 1 provides better approximations than Laplace. This requires evaluating the integrand at quadrature points, which is straightforward to implement in the existing C++ framework.
3. **Sparse automatic differentiation**: Recent work on sparsity-aware AD (Hill and Dalle 2025) could eliminate forward-difference overhead entirely by differentiating through the Cholesky factorisation implicitly.
4. **Communication-avoiding solvers**: For very large crossed random effects ($q > 50{,}000$), communication-avoiding Krylov methods could improve the PCG solver's performance on modern multi-core architectures.

7 Summary

fastmlm demonstrates that substantial speedups over lme4 are achievable through engineering optimisation rather than algorithmic change. The same profiled deviance, the same CHOLMOD factorisation, and the same Laplace approximation produce identical results 4–14times faster by eliminating overhead in formula parsing, R–C++ boundary crossing, sparse matrix construction, and BLAS linkage. The package is available on CRAN and requires no dependencies beyond Matrix and Rcpp.

8 Computational details

The results in this paper were obtained using R~4.5.3, fastmlm~0.1.0, lme4~1.1.38, and Matrix~1.7.4 on Ubuntu Linux with OpenBLAS (OpenBLAS 0.3.26 NO_LAPACKE DYNAMIC_ARCH NO_AFFINITY Haswell MAX_THREADS=64) [Haswell].

References

Bates, Douglas, Martin Mächler, Ben Bolker, and Steve Walker. 2015. "Fitting Linear Mixed-Effects Models Using lme4." *Journal of Statistical Software* 67 (1): 1–48. https://doi.org/10.18637/jss.v067.i01.

Hill, Adrian, and Guillaume Dalle. 2025. "Sparser, Better, Faster, Stronger: Efficient Automatic Differentiation for Sparse Jacobians and Hessians." https://arxiv.org/abs/2501.17737.